Flash Attention, Padding Lanes, and `lse_sum != lse_sum`

A controlled study of the NaNs nixnan reports in PyTorch's flash kernels

SC26 nixnan tutorial

`flash-attn-expt`, branch `Flash-Attn`

October 2026

The question

- nixnan on **BioMistral-7B** (4-bit NF4, HF + SDPA): millions of NaN/$\pm\infty$/subnormal operands
  - mostly in `pytorch_flash::flash_fwd_kernel`: `HMMA.16816.F32` op 2, `FADD [NaN,subnormal]`
  - almost nothing reaches memory
- Three candidate explanations:

H1: flash's guards

Flash filters NaN on purpose, e.g. in the combine kernel: `(lse_sum == 0 || lse_sum != lse_sum)`

H2: 4-bit

nixnan misreads packed NF4 weights as FP16/FP32 operands

H3: padding lanes

Fixed 128×128 tiles; rows/cols outside the sequence are never loaded or cleared

Test bed: TinyGPT on the same flash binaries

- Decoder-only, digits 0–9, 4 layers, $d = 128$, 4 heads $\times$ 32, RoPE
- 100 random training words (`training-set.txt`); loss 0.3939 (optimum 0.3938)
- FP32 linears (no tensor cores); attention FP16 via SDPA, **flash forced**
- Same venv as BioMistral: torch 2.6.0+cu124, bitsandbytes 0.48.1
- Task: 312 $\rightarrow$ 312452 (3 forward passes $\times$ 4 layers = 12 flash calls)
- A/B: 301-key prompt (2 KV splits) + an *external input* on the last embedding: A = 0, B = $+\infty$

Two lenses

1. nixnan (NVBit): training at `SAMPLING=32`, inference fully monitored, kernel whitelists

2. Source instrumentation: PyTorch 2.6's own flash source, recompiled with dump hooks, **bitwise-equal** to PyTorch's kernel

Lens 1 — nixnan: clean data reproduces BioMistral, 4-bit is innocent

run	SAMPLING	nixnan result
completion 312→6, every real tensor finite	0	`flash_fwd_kernel`: **1,183 FP32 + 184 FP16 NaN sites**
NF4 linears (`kDequantizeBlockwise`, `kgemm_4bit...`)	0	bitsandbytes: **0 NaN/Inf**, 19 subnormal sites
FP16 linears (cuBLAS/CUTLASS)	0	41 subnormal reports, 0 NaN/Inf
training, 1,500 steps	32	fwd: 1,493 sites; bwd: 458 sites, **no NaN**
prompt length $T \leq 16$ / $T \geq 32$	0	300–930 NaN sites / **0**
same $T = 16$ run, three times	0	681, 930, 937 sites

- Irregular in T, different on every repeat ⇒ looks like **uninitialized memory**
- nixnan's `INSTR_MEM` skips all of flash's stores (packed FP16 `STG.E.128`) ⇒ "no NaN in memory" is not evidence
- nixnan says *which instruction*, not *which lane*

Getting nixnan to run on PyTorch at all

- First direct runs: never past start-up (20+ min, 27 GB RSS). Three causes:

Instrumentation cost

NVBit spends ≈30 s per new cuBLAS/CUTLASS kernel.

Fix: `FUNCTION_WHITELIST` (full demangled names; a full tile is a different template instance, so log names first with a dummy whitelist)

Hangs

`F.embedding` hangs NVBit at load, even uninstrumented; so does the CUB radix sort in `weight[idx]`'s backward.

Fix: embedding as `one_hot(idx) @ W` (bit-identical)

LD_PRELOAD scope

`LD_PRELOAD=nixnan.so timeout ...` instruments `timeout` too: two writers, one `LOGFILE`.

Fix: `timeout N env LD_PRELOAD=... python`

Result: completion fully monitored in seconds; training (1,500 steps, `SAMPLING=32`) in 57 min.

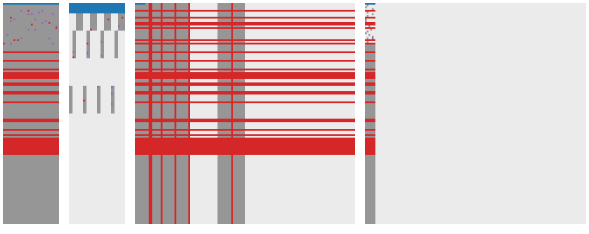
Lens 2 — instrumenting PyTorch's flash source

- Copy of `aten/.../cuda/flash_attn` (v2.6.0)
  - namespace renamed `instr_flash`; FP16 hdim-32 kernels only
  - host side = PyTorch's own `mha_fwd` (same split heuristic, tiles)
- Hooks (`instrumentation.patch`):
  - `flash_fwd_kernel`: whole tiles: `sQ sK sV` (raw FP16 bits), S, S masked, P, O, per-row `softmax.h` guard
  - split kernel: guard per (split, head, row)
  - combine: `lse_sum`, both disjuncts, replaced/stored value, NaN in O
- **16/16 bitwise equal** to `aten::_scaled_dot_product_flash_attention` (dumps on and off)

```
template <typename TensorS> __device__
void instr_dump_smem(float *dst,
                     TensorS const &s, int
    R, int C) {
  for (int i = threadIdx.x; i < R*C;
       i += blockDim.x)
    dst[i] = float(s(i / C, i % C).raw());
}
// ... in the masking loop:
if (instr_on && masking_step == 0) {
  instr_dump_smem(dst + dbg::Q, sQ, 128,
    32);
  instr_dump_smem(dst + dbg::K, sK, 128,
    32);
}
```

Fast math off: with `-use_fast_math` the combine LSE is 1 ulp off.

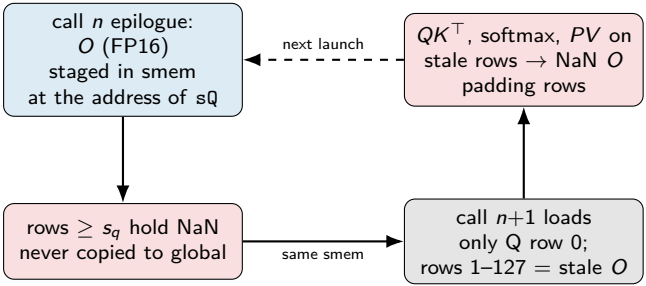
The padding lanes, dumped (last decode step, $s_q = 1$, $s_k = 6$)



call 11, 4 heads	values	NaN
Q row 0 (loaded)	128	0
Q rows 1–127 (never loaded)	16,256	**3,239**
S valid (row 0, keys 0–5)	24	0
S **padding rows**	65,024	**18,274**
P **padding rows**	65,024	**798**
O **padding rows**	16,256	**4,256**
O row 0 (the output)	128	0

- **Every NaN lies outside the sequence**
- `softmax.h`'s `sum != sum` fires on 133 padding rows; their LSE and O are never stored

Where the garbage comes from: the previous launch's output



- **416 of 508** stale Q rows are bit-identical to rows of the previous call's O tile
- Call n's 130 NaN O rows → call n+1's 130 NaN Q rows
- Stale K rows: zeros or old K rows
- Second identical run: **different** stale tiles, **bitwise-identical** output

Source: "We don't need to clear the sQ smem tiles since we'll only write out the valid outputs"

The guards, dumped: inputs A (clean) and B ($+\infty$)

12 calls, 4 heads	A	B
split `softmax.h`, real rows: `sum != sum` $\rightarrow -\infty$	0	**1,144**
split `softmax.h`, padding rows: `sum != sum`	1,588	3,931
combine guard fired (real rows)	0	**568**
via `lse_sum == 0`	0	568
via `lse_sum != lse_sum`	**0**	**0**
output rows with NaN	0	**752**
completion	..312489	..312000

- NaN is removed one level earlier: `softmax.h` turns a NaN sum into $\mathrm{lse}_s = -\infty$
- Combine sees only finite/$-\infty$; all $-\infty$ $\Rightarrow$ `lse_sum = 0`
- **The guard cleans the LSE, not** O: $0 \cdot \mathit{NaN}$ in $O \mathrel{+}= e^{\cdots} O_s^{\mathrm{acc}}$
- Causality broken by tiling: rows 256–300 share the poisoned split

Why `lse_sum != lse_sum` cannot fire for a positive scale

- `lse_sum` is NaN $\iff$ some $\mathrm{lse}_s \in \{NaN, +\infty\}$ $(\infty - \infty)$
- $\mathrm{lse}_s = (\ell{=}0 \vee \ell{\neq}\ell)\,?\,-\infty : m\sigma + \log \ell$
  $\Rightarrow$ needs $\ell = +\infty$
- Term $= 2^{\,\mathrm{fl}(x\sigma_2) - \mathrm{fl}(m\sigma_2)}$, $x \leq m$:
  - $\mathrm{fl}(m\sigma_2)$ finite: terms $\in [0, 1]$, $\ell \in [1, s_k]$
  - overflow: max element gives $\infty - \infty = NaN \rightarrow$ guard $\rightarrow -\infty$
- **Proposition**: $\sigma > 0 \Rightarrow$ unreachable for *every* FP16 input (incl. $\pm\infty$, NaN)
- TinyGPT $(1/\sqrt{32})$, BioMistral $(1/\sqrt{128})$: **impossible**

Source: `exp2f(x * scale - max_scaled)` "allows the compiler to use the ffma instruction" — **it doesn't**:

```
FMUL R28, R28, c[0x0][0x238] ;
...
FADD R154, -R47, R154 ;
FSETP.GEU.AND P1, PT, R160, -126, PT ;
MUFU.EX2 R160, R160 ;
```

At $\sigma = 10^{37}$: $\ell = NaN$ (unfused), PyTorch identical. A fused `FFMA` would give $\ell = +\infty$ and open the door.

Making it fire: a negative softmax scale

- $\sigma < 0$: term exponent $= (m - x)|\sigma_2| \geq 0$
- A masked score $x = -\infty$ gives $2^{+\infty} = +\infty \Rightarrow \ell = +\infty$
- $+\infty$ is neither 0 nor NaN: `softmax.h` lets it through, $\mathrm{lse}_s = +\infty$
- Combine: $L = +\infty$, $e^{\infty - \infty} \Rightarrow$ **`lse_sum != lse_sum`**
- SDPA does not validate the sign of `scale`

Raw call, $s_k = 301$, 2 splits (matches PyTorch, NaN-aware):

σ	split 1: ℓ, lse	`lse_sum != lse_sum`
$1/\sqrt{32}$	5.52, 4.73	0/4
10^{37}	1, $1.7 \cdot 10^{38}$	0/4
-0.1	$+\infty$, $+\infty$	**4/4**
-10	$+\infty$, $+\infty$	**4/4**

Inside TinyGPT, clean input A, `scale=-0.1`:

calls	!=	== 0
layer 0 (prefill + decode)	**1,208**	0
layers 1–3 (inputs already NaN)	0	3,636

Takeaways

- **BioMistral's flash NaNs are padding-lane arithmetic**, not model trouble
  - unloaded Q/K rows hold the previous launch's NaN-laden O; computed on, then discarded
  - non-deterministic counts, irregular in sequence length, nothing stored
- **4-bit is innocent**: bitsandbytes' NF4 kernels produce no NaN/Inf
- **Filtering happens in `softmax.h` and the $row < s_q$ store predicates**, not in the combine guard
  - `lse_sum != lse_sum`: provably dead for $\sigma > 0$; fires only with $\sigma < 0$ (or a fused-FFMA build + huge σ)
  - both guards protect the LSE, not the output; a real NaN in K/V crosses the causal mask
- **For nixnan**: lane-aware filtering of MMA operands beyond the sequence bound; classify packed FP16 stores in `INSTR_MEM`
- Source instrumentation, verified bitwise against the binary, complements binary monitoring

Everything (code, traces, report): `flash-attn-expt/` on branch `Flash-Attn`.